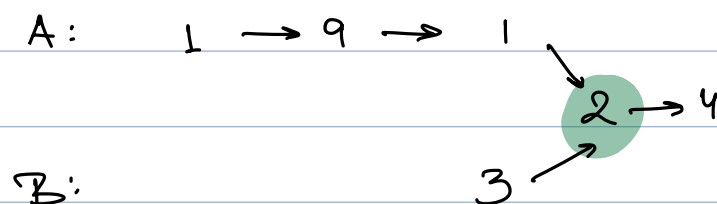
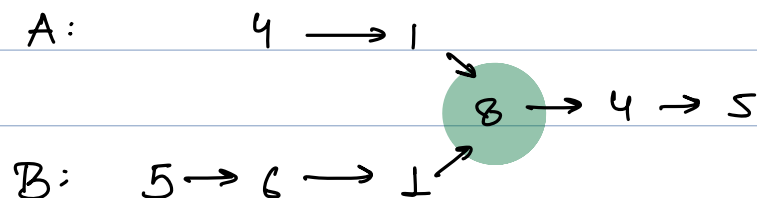


## Intersection of Two Linked Lists:

Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1:

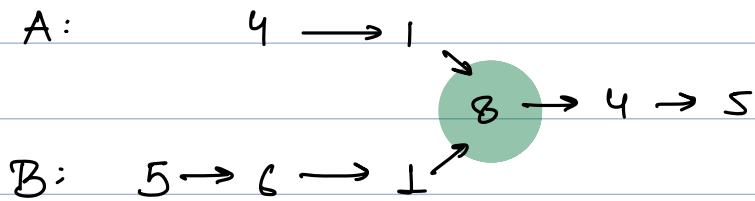


A: 2 → 6 → 4

B: 1 → 5

No intersection

## Optimized Approach



A and B do not at the same distance from intersection

$x$  = nodes before intersection in A

$y$  = nodes before intersection in B

$c$  = common part length

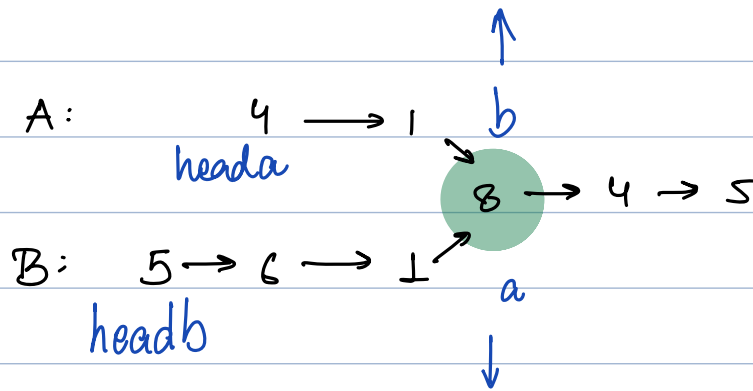
$$\Rightarrow A \text{ length} = x + c$$

$$B \text{ length} = y + c$$

We move pointers but they  $c$  at the same distance  
because  $x \neq y$

When a pointer reaches the end, send it to the head of the other list

$$\Rightarrow \begin{aligned} \text{Pointer } a &\Rightarrow x + c + y \\ \text{Pointer } b &\Rightarrow y + c + x \end{aligned}$$



```

ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    if (headA == null || headB == null)
        return null;

    ListNode a = headA;
    ListNode b = headB;

    while (a != b) {
        a = (a == null) ? headB : a.next;
        b = (b == null) ? headA : b.next;
    }

    return a;
}

```

TC:  $O(n+m)$   
 SC:  $O(1)$